

M V P

Model–View–Presenter Pattern

Presenter drives a passive view

DEFINITION

An MVC variant where a Presenter holds all UI logic and updates a passive View through an interface.

MVP cuts the view down to an interface of "show this / read that" and moves every decision into a Presenter that talks to the view only through that interface. Presenter and view are paired 1:1, and crucially the view holds no logic — so the screen's behaviour becomes plain objects you can test without rendering anything.

WHY IT MATTERS

UI is the hardest layer to unit-test — it wants a DOM, a device, a render loop. By making the view a passive interface and moving each decision into the presenter, that behaviour turns into ordinary objects a test can drive with a fake view, asserting exactly what the presenter asked the view to show.

— How it works

Model	Domain and data — the same role as in MVC, untouched by the UI.
View (passive)	A thin interface of show/read methods that a widget implements. Holds no logic; it forwards user events to the presenter and renders what it's told.
Presenter	Holds all UI logic. Reads the model, decides, and calls the view through its interface. One presenter per view, holding a reference to it.

HOW TO APPLY

1. Define the view as an interface of intent: showError, setRows, getInput.
2. Move every decision out of the widget and into the presenter.
3. Hand the presenter its view through a field, not a newwed instance, so a fake can replace it.
4. Unit-test the presenter with a stub view, asserting the calls it makes.

LITMUS TEST

Could you exercise this screen in a unit test with a fake view and assert what the presenter called? If logic is welded to widgets, it isn't MVP yet.

— Smell → fix

A component where the widget itself decides, validates, and calls the API:

```
class LoginComponent {
  onSubmit() {
    if (!this.emailEl.value) this.errorEl.show() // view decides
    else this.api.login(this.emailEl.value) // view calls API
  }
} // logic welded to widgets: untestable without a DOM
```

↓ MOVE LOGIC TO THE PRESENTER

```
interface LoginView { showError(m: string): void }
class LoginPresenter {
  private view!: LoginView // injected, not newed
  submit(email: string) {
    if (!email) return this.view.showError('required')
  }
} // plain logic; the view is a passive interface
```

The presenter never touches a widget — only the LoginView interface, handed in via a field. A unit test assigns a fake view and asserts showError was called; production passes the real component. This is MVP's "Passive View" flavour.

USE WHEN

- ✓ Testable UI (classic Android, WinForms)
- ✓ Views that are hard to unit-test

AVOID WHEN

- ▲ Simple views where binding (MVVM) is enough

— Trade-offs & pitfalls

PROS

- + Highly testable presenter
- + The view is dumb

CONS

- Presenter can grow large
- More interfaces / boilerplate

COMMON PITFALLS

- ▲ The presenter swells into a god object as all logic funnels through it.
- ▲ Leaking widget types (DOM nodes, Android Views) into the presenter — it should see only the interface.
- ▲ Interface bloat: a giant view contract that's painful to stub in tests.

WHERE YOU SEE IT

- ▶ Classic Android (pre-ViewModel), WinForms, GWT — the canonical MVP homes.
- ▶ Legacy front-ends retrofitted for testability without a binding framework.
- ▶ Any view that's hard to unit-test, wrapped behind a presenter + interface.

SEE ALSO

MVC

MVP tightens MVC: the view goes fully passive and the presenter replaces the controller 1:1

MVVM

swap the manual presenter-to-view calls for declarative data binding

DIP

the presenter depends on a LoginView abstraction, not a concrete widget

Dependency Injection

the view is handed to the presenter from outside — exactly how you inject a fake in tests

— Interview & sources

How does MVP differ from MVC?

In MVP the view is fully passive with a 1:1 link to its presenter, which holds all UI logic. In MVC the controller handles input and the view may observe the model directly.

Why is MVP called testable?

The presenter talks to the view only through an interface, so a unit test substitutes a fake view and asserts the calls — no rendering engine needed.

Passive View vs Supervising Controller?

Two MVP flavours. In Passive View the view has zero logic — the presenter sets every value, maximum testability. In Supervising Controller the view does simple declarative binding itself and the presenter handles only the complex logic.

What is MVP's main downside?

The presenter tends to absorb everything and grow large, and every view needs an interface — more boilerplate than binding-based MVVM.

REMEMBER

The view is dumb on purpose — it exposes hooks, the presenter pulls the strings.

SOURCES

Martin Fowler — "GUI Architectures": Passive View & Supervising Controller

Mike Potel — "MVP: Model-View-Presenter" (Taligent/IBM, 1996)

Classic Android, WinForms, GWT — MVP for testable UI